Totalvurdering som oppdateres med en gang

Hvordan spillsiden ber DiceRating hente den nye totalen etter at et terningkast er lagret, med @ref

Når brukeren lagrer et terningkast på spillsiden, skal totalvurderingen i `DiceRating` vise den nye verdien med en gang, uten at siden lastes på nytt. Løsningen er at spillsiden, etter lagringen, ber akkurat den `DiceRating` som vises om å hente totalen på nytt.

To begreper som må holdes fra hverandre

	Tjeneste, for eksempel `GameNightState`	**Komponent, for eksempel** `DiceRating`
Lages av	Dependency injection, når noen ber om den med `@inject`	Blazor, hver gang `<DiceRating />` står i markupen
Antall	Én per scope. Alle som injiserer den, får samme objekt	Én per sted den står. To steder gir to instanser
Registreres i `Program.cs`	Ja, med `AddScoped` eller `AddSingleton`	Nei
Nås fra en annen komponent	Med `@inject`	Med `@ref`, eller med parametere og `EventCallback`

Kjernen: et varsel med `OnChange` virker for en tjeneste fordi alle deler samme objekt. En komponent som vises på siden, kan ikke hentes med `@inject`. Skal forelderen snakke med akkurat den komponenten den viser, trenger den en referanse til den, og det er det `@ref` gir.

Hvem må si fra til hvem?

Det er spillsiden som vet at et nytt terningkast er lagret. Det er `DiceRating` som trenger å få vite det, fordi den viser totalen. Beskjeden går derfor fra forelder til barn. Derfor kaller spillsiden en metode på `DiceRating`, i stedet for at `DiceRating` varsler spillsiden.

Endring 1: DiceRating får en metode som kan kalles utenfra

Hentingen av totalen flyttes til en egen `public` metode. Den kalles når komponenten lages, og kan kalles igjen av forelderen senere.

Components/Shared/DiceRating.razor, @code

```
protected override async Task OnInitializedAsync() => await RefreshTotalAsync();

// Called by the parent after a new rating is saved
public async Task RefreshTotalAsync()
{
    try
    {
        totalRating = await ApiClient.GetTotalRatingByGame(BoardGame!.Id);
        error = null;
    }
    catch
    {
        error = "Henting av total vurdering feilet";
    }

    StateHasChanged();
}
```

- `public` gjør at forelderen kan kalle metoden.
- `error = null` fjerner en gammel feilmelding når hentingen lykkes.
- `StateHasChanged()` tegner komponenten på nytt. Når metoden kalles av forelderen, og ikke av Blazor selv, vet ikke Blazor at noe er endret uten dette kallet.

Endring 2: spillsiden tar vare på en referanse

`@ref` på komponenten lagrer en referanse til instansen i et felt i forelderen.

Components/Pages/GamesDetails.razor, markup

```
<DiceRating @ref="diceRating" BoardGame="game" @key="game.Id" ReadOnly="false" />
```

Components/Pages/GamesDetails.razor, @code

```
// The DiceRating component rendered on this page
private DiceRating? diceRating;
```

Feltet er `null` til komponenten er tegnet første gang. Derfor er typen nullbar, og det sjekkes før bruk.

Endring 3: spillsiden ber om ny total etter lagring

```
Components/Pages/GamesDetails.razor, @code

private async Task UpdateRating()
{
    try
    {
        await ApiClient.UpdateRatingAsync(Id, new UpdateRatingRequest { Rating = game!.Rating
    });

        // Fetches the new total in the DiceRating shown on the page
        if (diceRating is not null)
        {
            await diceRating.RefreshTotalAsync();
        }
    }
    catch
    {
        error = "Oppdatering av vurdering feilet";
    }
}
```

Rekkefølgen er viktig. Totalen hentes **etter** at lagringen er ferdig. Hentes den før, regner API-et fortsatt ut totalen uten det nye terningkastet.

Hele flyten

```
Bruker klikker «Lagre vurdering»
  → GamesDetails.UpdateRating()
      → GameApiClient.UpdateRatingAsync()     PUT /api/games/{id}/rating
      ← 204 No Content
      → diceRating.RefreshTotalAsync()        (instansen som vises på siden)
          → GameApiClient.GetTotalRatingByGame()   GET /api/games/{id}/totalRatingByGame
          ← ny total
          → StateHasChanged()                 DiceRating tegnes på nytt
```

Rydding

- `DiceRating` er ikke lenger registrert som tjeneste i `Program.cs`, og spillsiden injiserer den ikke.
- `DiceRating` trenger ingen egen `OnChange`-hendelse.
- Spillhylla er uendret. Hvert `GameCard` henter fortsatt sin egen total når siden åpnes.

Andre måter å løse det på

Løsning	Slik virker den	Passer når
`@ref` (valgt her)	Forelderen kaller en metode på barnet	Én bestemt forelder må be ett bestemt barn om å gjøre noe
Parameter	Forelderen henter totalen og sender den ned som `[Parameter]`	Forelderen eier dataene, «data ned». `DiceRating` trenger da ikke API-et
Tilstandstjeneste med `OnChange`	En tjeneste holder vurderingene og varsler alle som lytter	Flere komponenter på ulike steder skal oppdateres samtidig

Kontrollert

- Prosjektet kompilerer med endringene.
- API-et er startet og testet: totalen for spill 1 går fra 0 til 6 etter terningkast 6, og til 4 etter terningkast 3. Snittet 4,5 rundes ned fordi totalen er en `int`.
- **Ikke kontrollert:** at visningen i nettleseren oppdateres. Lagre et terningkast på spillsiden, og se at totalen endres uten ny innlasting.

Hva det første forsøket bommet på

Det første forsøket brukte riktig idé, et varsel med `OnChange` slik `GameNightState` gjør, men på feil objekt og i feil retning. Tre ting gjorde at totalen ikke oppdaterte seg:

1. **Komponenten ble behandlet som en tjeneste.** `DiceRating` var registrert med `AddScoped` og hentet inn med `@inject`. Det ga et eget objekt som aldri ble tegnet. Spillsiden lyttet derfor på en annen `DiceRating` enn den som vistes på siden.
2. **Varselet gikk feil vei.** `DiceRating` varslet når den selv hadde hentet totalen. Men det var spillsiden som visste at et nytt terningkast var lagret, og `DiceRating` som trengte beskjeden.
3. **Totalen ble bare hentet én gang.** Hentingen lå i `OnInitializedAsync`, som kjører når komponenten lages, og ikke igjen etter lagring.

Lærdom: varsler med `OnChange` virker for tjenester, fordi alle deler samme objekt. Skal en forelder nå en bestemt komponent den viser, brukes `@ref` eller parametere.